

工业园区智能可视化与 AI 平台 · 技术方案 / 架构 / 功能设计 (zh-CN)

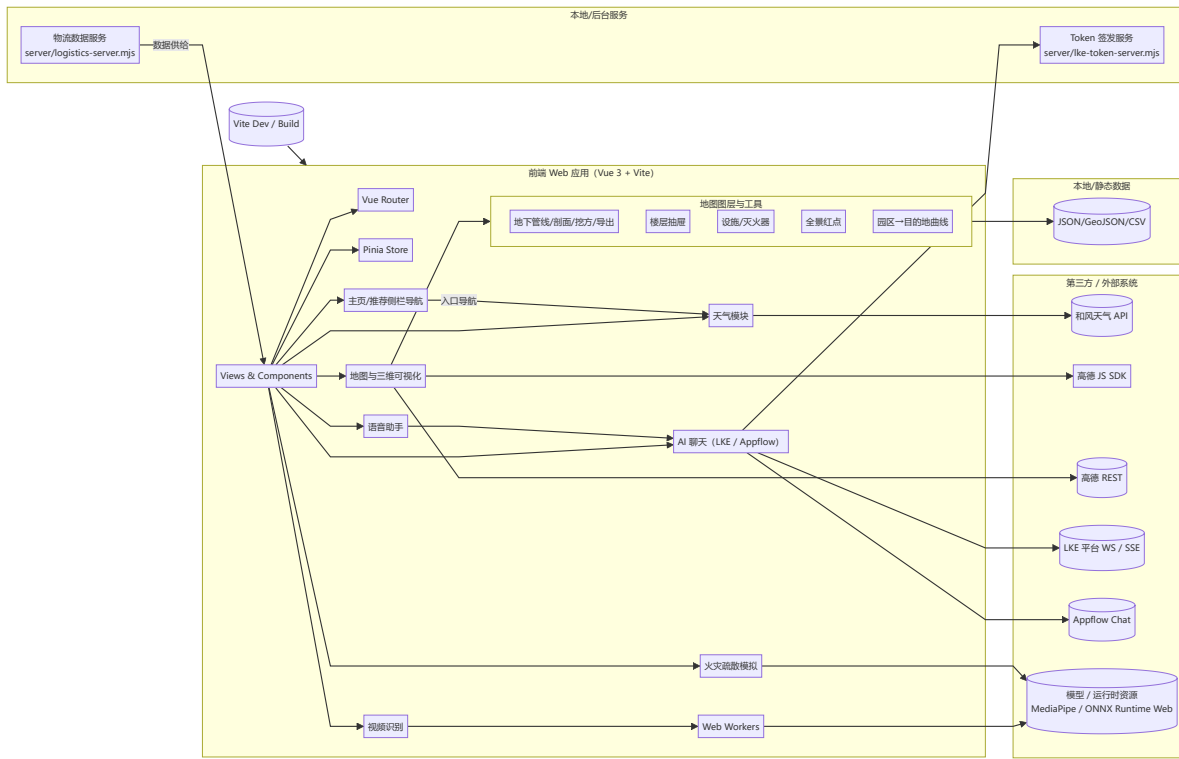
执行摘要 (Executive Summary)

本方案面向工业园区的智能可视化与 AI 一体化场景，采用 Vue 3 + Vite 的前端架构与轻量服务配合，强调“端上智能、弱网可用与安全合规”。核心优势：

- 前端 AI 集成：基于 MediaPipe 与 Web Speech API，在浏览器端实现轻量视频识别与语音交互，降低服务端压力并提升交互即时性。
- 数据可视化：融合高德地图与天气数据，支持 2D/3D 图层、风险着色与路线避险，为调度与决策提供直观依据。
- 灵活 AI 聊天：以 LKE 为主、Appflow 为备的双通道策略，既保证深度集成与稳定性，也兼顾快速接入与容灾降级。

同时，本架构兼顾性能（按需加载、GPU/CPU 回退）、安全（密钥后端签发）与弱网可用性（脚本动态加载与兜底），满足演示与生产落地的双重要求。

系统架构图 (System Architecture)



注：

- 前端仅注入以 `VITE_` 开头的环境变量；真实后端密钥仅在服务端（如 `TokenSrv`）使用。
- LKE 优先用 WS / SSE，Appflow 用于备选与独立连通性验证。
- 视频识别优先采用 MediaPipe Tasks，必要时可切换到 ONNX Runtime Web (WASM/Threads/WebGL)。

1. 背景与目标

- 为园区监控、态势感知与决策辅助提供可视化与 AI 能力：
 - 天气风险分析与路线避险
 - 语音助手与 AI 聊天联动
 - 视频目标识别与统计
 - 地图场景与多图层叠加
 - 火灾疏散模拟与多视角播放
- 交付要求：易演示、易排错、可扩展、安全合规。

2. 技术选型

2.1 选型总览

- 前端：Vue 3、Vite、TypeScript、Pinia、Vue Router
- 可视化/地图：高德 JS SDK (AMap)，3D 图层（项目内封装）
- AI / 多媒体：Web Speech API (STT/TTS)、MediaPipe Tasks (优先)、ONNX Runtime Web (备选)
- 聊天：LKE (主) / Appflow Chat (备)
- 服务：Node.js 轻量服务 (Token 签发、物流数据)
- 数据：公共 API (和风天气、高德 REST) + 本地 JSON/GeoJSON

2.2 选型考量与备选方案 (Alternatives Considered)

- 视频识别：
 - 选型：MediaPipe Tasks (优先)
 - 理由：自动 GPU/CPU 回退，浏览器端部署成本低，API 友好，生态活跃。
 - 备选：ONNX Runtime Web (WASM/WebGL/Threads)。适合需要加载特定 ONNX 模型的场景，但初始化与线程/显存管理更复杂，包体与首帧开销较大。
- 聊天服务：
 - 选型：LKE (主) / Appflow (备)
 - 理由：LKE 提供 WS/SSE 流式接口，便于深度集成与实时交互；Appflow SDK 接入快、配置简单，适合作为连通性验证与降级兜底路径。
 - 取舍：优先保证主链路稳定 (LKE)，保留备链路以应对网络与配额波动。

3. 功能设计 (Feature Design)

3.1 天气分析与路线避险 (Weather)

- 输入：起点/终点、路径规划、天气指标 (温度/风/降水/能见度)
- 处理：省份/路段风险评估，融合预警事件
- 输出：
 - 省份风险着色 (低/中/高/极端)
 - 路线评分与替代建议

- Tooltip 详情/预警点位
- 关键交互：图层切换、路线选择、点位详情
- 主要文件：src/services/weather.ts、src/views/WeatherAnalysis.vue

3.2 语音助手 (Voice)

- 能力：开始/停止识别、TTS 播报、命令解析（页面跳转、图层、全屏等）
- 联动：将最终识别文本发送至聊天模块，支持语音→AI→播报闭环
- 主要文件：src/composables/useVoiceAssistant.ts、src/utis/voiceCommands.ts、src/components/VoiceAssistantFloat.vue

3.3 聊天助手 (Chat)

- LKE (主)：WS/SSE 连接、Token 服务签发、流式回复、互斥控制与 TTS 播报
- Appflow (备)：SDK 初始化、显隐控制、发送消息、独立测试页验证
- 主要文件：src/composables/useAppflowChat.ts、server/lke-token-server.mjs、测试页 tests/pages/*

3.4 视频识别 (Video Recognition)

- 优先 MediaPipe Tasks（自动 GPU/CPU 回退），对性能不足环境提供 Mock 兜底
- 输出：检测框、置信度曲线、统计面板
- 主要文件：src/views/VideoRecognition.vue、src/workers/*、public/models/、public/onnxruntime-web/

3.5 地图与三维可视化 (Map / Scene)

- 动态加载 AMap 脚本与插件、弱网与错误兜底
- 路线规划与覆盖物渲染、图层控制
- 主要文件：src/utis/AmapLoader.ts、docs/modules/map-amap-fix.md

图层与工具清单

- 地下管线（含地形透明度滑块）
 - 剖面分析：依次两点生成剖面，缓冲距离可调；附近管线列表；高亮闪烁；导出 CSV/GeoJSON
 - 挖方分析：多边形绘制（单击加点、双击/按钮完成、Backspace 撤销、Esc 取消）；面积/周长标注；附近管线列表
 - 图例：按管线类型分组显隐（与数据源 show 状态同步）
- 楼层抽屉：点击楼层模型展开目标楼层、其他楼层半透明，再次点击空白复位
- 设施与灭火器：提高相对地面高度与标签气泡样式，强化可视性
- 全景红点：自定义图标、聚合强度可调、距离显示；点击打开全景查看器
- 园区→目的地曲线：数量上限/分段/高度比例可调，展示辐射范围
- 性能调优：Tiles 细节（SSE）开关与参数

3.6 主页与商家推荐侧栏（入口导航）

- 能力：侧栏筛选/排序、主视图高亮/飞行定位、业务入口跳转（天气分析）
- 交互：从首页侧栏点击“天气分析”进入 `/weather`，体现“从业务到分析”的链路
- 关联：与地图图层联动（设施、灭火器、全景红点、地下管线、楼层抽屉、园区曲线等）
- 主要文件：`src/components/MapView.vue`（首页场景核心逻辑）、路由 `src/router/index.ts`

3.7 火灾疏散模拟（Fire Evacuation）

- 能力：视频网格→播放器，播放/暂停、静音、加载指示、视角切换（保持当前进度比例）
- 资源：`public/FireEvacuation/` 下的演示视频（多视角）
- 主要文件：页面 `src/views/FireEvacuation.vue`，路由 `/fire-evacuation`

4. 核心 API 与数据契约（Contracts）

4.1 Weather Service（示例）

- `getProvinceWeather(): Promise<ProvinceWeather[]>`
- `analyzeRoute(params: { origin, destination }): Promise`
- 错误：无 Key / API 限频 / 网络异常 → UI Toast + 重试

Request（示例）

```
{ "origin": "116.40,39.90", "destination": "121.47,31.23" }
```

Response（示例）

```
{
  "routeId": "RT_20251028_001",
  "score": 45.5,
  "riskLevel": "High",
  "suggestion": "建议延后2小时出发或改道G2",
  "riskSegments": [
    { "location": "...", "event": "大风预警", "level": "High" }
  ]
}
```

4.2 Voice → Chat 桥接

- 事件：`finalText`（最终文本）→ `chat.sendMessage(text)`
- 并发控制：语音/播报/AI 回复互斥锁，避免抢占音频设备
- 路由口令：解析“打开天气页面”“打开火灾疏散”“全屏/退出全屏”“显示/隐藏天气图层”等，并触发对应处理（见 `src/utils/voiceCommands.ts`）

4.3 Chat (LKE)

- 初始化：基于 `VITE_LKE_*` 与 `/getDemoToken`
- 协议：WS/SSE (心跳/重连/超时处理)
- 降级：不可用时提示并提供独立测试页排查

4.4 Token 服务 (server/lke-token-server.mjs)

- GET `/getDemoToken` → { token, expireAt }
- 安全：仅服务器持有 `SECRET_ID/SECRET_KEY`，前端不暴露
- 可加：Referer 限制、频率限制、审计日志

5. 部署与环境 (Dev/Prod)

- 环境变量：仅 `VITE_` 变量注入前端；服务器端私密变量不进入构建
- 本地开发：`npm run dev`，独立测试页通过 `/tests/pages/*.html` 访问
- 构建产物：`npm run build` 输出至 `dist/`
- 资源托管：地图/模型/ORT 的静态资源需确认路径与跨域策略
 - 弱网/离线：AMap 按需/延迟加载与兜底策略，详见 `AMAP_API_FIX.md` 与 `docs/modules/map-amap-fix.md`

6. 安全与合规

- 密钥管理：`.env.example` 提供占位；`.env.local` 不提交；服务器持私密变量
- 平台侧限制：AMap/QWeather/Appflow/LKE 配置来源域名/频率/白名单
- 网络安全：Token 服务增加速率限制与 Referer 校验

7. 非功能性需求 (NFR)

7.1 性能 (Performance)

- `/weather` 首屏性能：在非弱网环境下 LCP < 3s (按需加载脚本与数据缓存)。
- 视频识别 (MediaPipe)：在目标设备 (如 i5 笔记本) > 8 FPS；设备不足时自动回退 CPU，再不足回退 Mock，确保 UI 不卡顿。
- 地图与天气：AMap 脚本/插件按需加载，省份数据缓存，仅在图层变更时重绘。

7.2 可用性与韧性 (Availability & Resilience)

- 弱网 (Slow 3G) 下地图 10s 内可加载；若失败显示重试与兜底提示。
- AI 聊天 (LKE) 断线后 30s 内自动重连；必要时回退 SSE。
- Weather/AMap API 限频或错误：前端 Toast + 指数退避重试；必要时走本地缓存。

7.3 可维护性 (Maintainability)

- 关键模块提供独立测试页 (Chat、Voice 等) , 便于隔离排错与回归。
- 统一日志与埋点: 初始化耗时、连接状态、推理 FPS、错误率。

7.4 兼容性与边界 (Compatibility & Constraints)

- 浏览器能力差异: 语音/WebAssembly/Workers 在低端或受限环境下功能降级或禁用。
- 公共 API 稳定性与配额受限: 必要时引入服务端代理与缓存。
- 离线/弱网: 静态资源与脚本懒加载、可选离线包、重试与提示。

8. 可观测与排错

- 日志: 独立测试页 (Appflow、语音) 提供实时日志与调试入口
- 指标: 初始化耗时、WS/SSE 连接状态、推理 FPS、错误率
- FAQ: 集中于 docs/OPERATION_GUIDE.zh-CN.md 与 docs/troubleshooting/*

10. 附：独立测试页索引

页面/路径	作用	备注
/tests/pages/appflow-test-standalone.html	Appflow SDK 最小验证 (初始化/显隐/发送/日志)	联通性与降级兜底验证
/tests/pages/appflow-test.html	Appflow 在项目态方法与事件	接口回调与显隐控制
/tests/pages/voice-ai-integration.html	语音 + Chat 联动闭环	识别→AI 回复→TTS 播报
/tests/pages/voice-debug.html	语音识别调试	实时日志、环境/权限排查
/tests/pages/test-tts.html	TTS 参数与发音对比	语速/音调/声音切换